



**CEBU INSTITUTE OF TECHNOLOGY**  
**U N I V E R S I T Y**

# IT342-G2

## System Integration and Architecture

---

### **System Design Document (SDD)**

---

Project Title: Paluto

Prepared By: Auditor, Mechole Angelou M.

Version: 0.5

Date: March 2, 2026

Status: Final

## REVISION HISTORY TABLE

| Version | Date      | Author           | Changes Made                                                                    | Status   |
|---------|-----------|------------------|---------------------------------------------------------------------------------|----------|
| 0.1     | 2-23-2026 | Auditor, Mechole | Initial draft                                                                   | Final    |
| 0.2     | 2-28-2026 | Auditor, Mechole | Initial UI/UX designs of Web and mobile application                             | Review   |
| 0.3     | 3-1-2026  | Auditor, Mechole | Initial draft of section 5, 6, and 8. Revisions" "Client" changed to "Customer" | Review   |
| 0.4     | 3-2-2026  | Auditor, Mechole | Additions: Journey 1<br>Revisions: Journey numbering. API Endpoints             | Review   |
| 0.5     | 5-26-2026 | Auditor, Mechole | Updates endpoints, diagrams, and journey.                                       | Review   |
| 0.6     | [Date]    | [Your Name]      | Final review and corrections                                                    | Final    |
| 1       | [Date]    | [Your Name]      | Baseline version for development                                                | Approved |

## TABLE OF CONTENTS

### Contents

|                                           |    |
|-------------------------------------------|----|
| EXECUTIVE SUMMARY                         | 4  |
| 1.0 INTRODUCTION                          | 5  |
| 2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION | 5  |
| 3.0 NON-FUNCTIONAL REQUIREMENTS           | 8  |
| 4.0 SYSTEM ARCHITECTURE                   | 9  |
| 5.0 API CONTRACT & COMMUNICATION          | 10 |
| Authentication Endpoints                  | 11 |
| User Registration                         | 11 |
| User Login                                | 11 |
| 6.0 DATABASE DESIGN                       | 13 |
| 7.0 UI/UX DESIGN                          | 14 |
| 8.0 PLAN                                  | 17 |

# EXECUTIVE SUMMARY

## 1.1 Project Overview

Paluto is a specialized booking platform designed to bridge the gap between professional home cooks and busy families or professionals. Unlike standard food delivery apps, this system facilitates on-site meal preparation services, allowing customers to have personalized meals cooked directly in their own kitchens. The system includes a Spring Boot backend API, React web application, and Android mobile app, all integrated to provide a seamless appointment booking across platforms.

## 1.2 Objectives

1. Develop a fully functional Service-Booking platform with authentication, multi-user roles, cook portfolios, appointment booking scheduling, and service rating.
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile)
3. Create RESTful APIs for communication between all system components
4. Design a responsive user interface that works consistently across web and mobile platforms
5. Deploy all system components to production-ready environments

## 1.3 Scope

### Included Features:

- User registration and authentication (email/password)
- Google login integration
- Cook portfolio (add, remove, update certificates, menu, specialties, and rates)

- Transactional Appointment booking system
- Cook Service Rating System
- Payment Gateway Integration (sandbox)
- Email notification system
- In-app notifications
- Admin panel
- Native Android mobile application
- PostgreSQL database with relational schema

**Excluded Features:**

- Push notifications
- Advanced analytics dashboard

## 1.0 INTRODUCTION

### 1.1 Purpose

This document serves as the comprehensive design specification for the Paluto Booking system. It provides detailed requirements, architectural decisions, API contracts, database design, and implementation roadmap to guide development and ensure all components integrate seamlessly.

## 2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

### 2.1 Project Overview

**Project Name:** Paluto

**Domain:** Service-Booking platform

**Primary Users:**

1. Customers
2. Cooks
3. Administrator

**Problem Statement:** Busy professionals and families struggle to maintain healthy eating habits due to lack of time, often relying on unhealthy fast food.

**Solution:** A specialized booking platform that connects these individuals with home cooks for on-site meal preparations.

## **2.2 Core User Journeys**

### **Journey 1: Cook portfolio management**

1. Cook register account and setup profile (hourly rate, years of experience, bio)
2. Navigate to Portfolio tab
3. Go to menu sub tab
4. Click “add” button
5. Specify menu title, description, ingredients list, ingredients cost, image, prep time.
6. Confirms menu addition and Click “publish” button

### **Journey 2: First-time customer booking a service**

1. Customer opens the web or mobile application
2. Clicks “Sign Up” and creates an account
3. Customer logs in with the registered account
4. Browse cook profiles to view menus and hourly rates
5. Selects a cook clicks “view menu”
6. Selects menu
7. Book menu with selected quantity
8. Redirect to payment page
9. Selects specific date, time, and address
10. Confirms the booking and clicks “Pay now” button
11. Receives a booking confirmation once the cook accepts booking
12. After the service, the customer rates the cook.

### **Journey 3: Cook accepts a booking**

1. Cook registers and completes their profile with certification/ specialties
2. Sets hourly rate and uploads specialty menus
3. Receives a notification for a new booking request
4. Navigates to “My bookings” tab
5. Click booking request and review details and confirm the schedule
6. After the service, the cook marks the appointment as completed.

### **Journey 4: Admin manages users**

1. Admin logs into the web or mobile application
2. Navigates to “users” section
3. View user details

## **2.3 Feature List (MoSCoW)**

### **MUST HAVE**

1. User authentication (register, login, logout)
2. Role based access (customer, cooks, admin)
3. Cook portfolio (certification, menus, rates)
4. Appointment booking (date, time, menu, cook)

### **SHOULD HAVE**

1. Cook Service rating
2. Booking history
3. Google login integration
4. Payment gateway integration

### **COULD HAVE**

1. Favorite Cooks
2. Cook filtering
3. Cook dashboard
4. Email notification
5. In app notification
6. Responsive design for all screen sizes

### **WON'T HAVE**

1. Real payment processing
2. Advanced analytics
3. Multi-language support
4. Real-time GPS tracking of the cook

## **2.4 Detailed Feature Specifications**

### **Feature: User Authentication**

- **Screens:** Customer Registration, Customer Login, Cook Registration, Cook Login, Forgot Password
- **Fields:** First name, Last name, address, phone number, Email, Password, Confirm Password, cook( hourly rate, years of experience, bio)

- **Validation:** Email format, password strength, uniqueness, phone number format,
- **API Endpoints:** POST /customer/register, POST /cook/register, POST /auth/login
- **Security:** JWT tokens, password hashing with bcrypt

#### Feature: Cook portfolio

- **Screens:** Portfolio edit (Cook)
- **Display:** Menu, rate, certification/s, profile
- **Functions:** Add/edit menu, view ratings/reviews, upload/ remove certifications.
- **API Endpoints:** GET /certificates/my-certificates, GET /certificates/upload, GET /cook/{id}/services, GET /cook/my-services, PUT /services/{id}, GET /cook/{cookId}/average,

#### Feature: Appointment Booking

- **Screens:** Booking form, Appointment Summary, Booking History
- **Functions:** Select a cook, choose schedule (date & time), select menu, submit booking request, accept booking
- **Data Collected:** Service address, date/time, and menu.
- **API Endpoints:** POST /bookings, GET bookings/{id}, PUT /bookings/{id}/status

#### Feature: Service Rating

- **Screens:** Post-service Rating form, Cook profile rating section
- **Fields:** Star rating(1-5)
- **Persistence:** Ratings are linked to the cook's profile and appointment.
- **API Endpoints:** POST /ratings, GET /ratings/cook/{cookId}

## 2.5 Acceptance Criteria

### AC-1: Successful User Registration

- Given I am a new user
- When I enter valid firstname, lastname, phonenumber, address, email and strong password
- And confirm password matches
- And click "Create Account"
- Then my account should be created
- And I should be redirected to login page

### AC-2: Cook Portfolio Creation

- Given I am logged in as a cook



- When I navigate to my portfolio dashboard and add/edit my specialty menus, hourly rates, and certifications.
- And I save the changes
- Then the updated information should be visible to customers when they search for cooks
- And the certifications should be stored and available for Admin verification.

### **AC-3: Appointment Booking Flow**

- Given I am logged in as a "Customer"
- When I select a cook's profile and choose an available date and time slot.
- And I provide the service location/address and confirm the booking request.
- Then the system should create a "Pending" appointment record.
- And the cook should see the new request in their management dashboard.

## **3.0 NON-FUNCTIONAL REQUIREMENTS**

### **3.1 Performance Requirements**

- API response time:  $\leq 2$  seconds for 95% of requests
- Web page load time:  $\leq 3$  seconds on broadband
- Mobile app cold start:  $\leq 5$  seconds
- Support 30 concurrent users
- Database queries complete within 500ms

### **3.2 Security Requirements**

- HTTP for all communications
- JWT token authentication
- Password hashing with bcrypt
- Role-based access control
- SQL injection prevention
- XSS protection
- Rate limiting: 100 requests/minute per IP
- Admin endpoints require role verification

### **3.3 Compatibility Requirements**

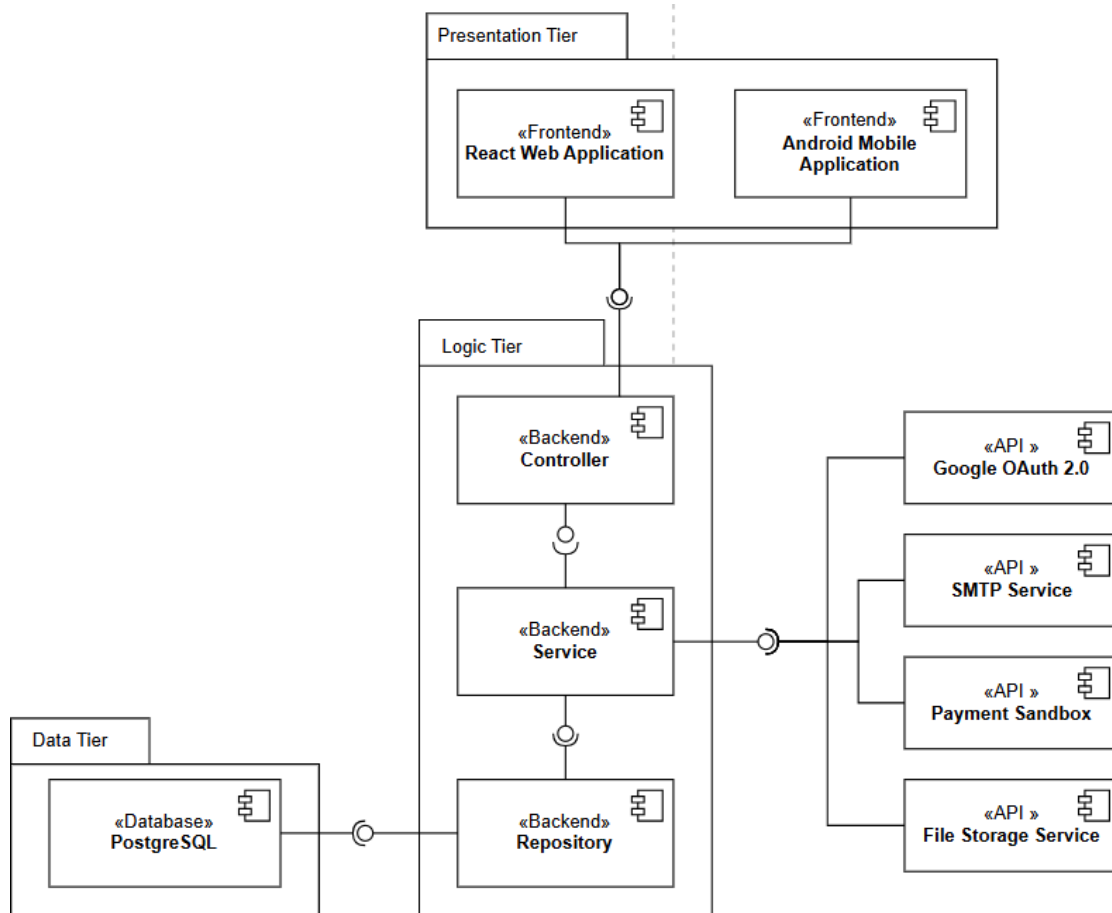
- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 34 (Android 14)

### **3.4 Usability Requirements**

- New users should be able to complete their first cook booking within 5 minutes.
- Consistent navigation across all pages
- Clear error messages with recovery options

## 4.0 SYSTEM ARCHITECTURE

### 4.1 Component Diagram



#### Technology Stack:

- **Backend:** Java 21, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Retrofit,
- **External Integrations:** Google OAuth, Supabase storage/buckets, Paymongo (sandbox)
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

## 5.0 API CONTRACT & COMMUNICATION

### 5.1 API Standards

|                           |                                                                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Base URL</b>           | https://[server_hostname]:[port]/api/                                                                                                                                 |
| <b>Format</b>             | JSON for all requests/responses                                                                                                                                       |
| <b>Authentication</b>     | Bearer token (JWT) in Authorization header                                                                                                                            |
| <b>Response Structure</b> | <pre>{   "success": boolean,   "data": object null,   "error": {     "code": string,     "message": string,     "details": string,   },   "timestamp": string }</pre> |

### 5.2 Endpoint Specifications

#### Authentication Endpoints

##### Customer Registration

|                            |                                                                                                                                                                                                                                        |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Description</b>         | User Registration                                                                                                                                                                                                                      |
| <b>API URL</b>             | /customer/register                                                                                                                                                                                                                     |
| <b>HTTP Request Method</b> | POST                                                                                                                                                                                                                                   |
| <b>Format</b>              | JSON for all requests/responses                                                                                                                                                                                                        |
| <b>Authentication</b>      | None                                                                                                                                                                                                                                   |
| <b>Request Payload</b>     | <pre>{   "email": &lt;email&gt;,   "password": &lt;password&gt;,   "address":&lt;address&gt;   "firstname": &lt;firstname&gt;,   "lastname": &lt;lastname&gt;, }</pre>                                                                 |
| <b>Response Structure</b>  | <pre>{   "success": boolean,   "data": {     user {       "email": &lt;email&gt;,       "firstname": &lt;firstname&gt;,       "lastname": &lt;lastname&gt;,       "address": &lt;address&gt;,       "role": customer     }   } }</pre> |

|  |                                                                                                                                                                                 |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre> }, "accessToken": &lt;token&gt;, "refreshToken": &lt;token&gt;, }, "error": {   "code": string,   "message": string,   "details": string, }, "timestamp": string } </pre> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Cook Registration

|                            |                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Description</b>         | User Registration                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>API URL</b>             | /cook/register                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>HTTP Request Method</b> | POST                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Format</b>              | JSON for all requests/responses                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Authentication</b>      | None                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Request Payload</b>     | <pre> {   "email": &lt;email&gt;,   "password": &lt;password&gt;,   "address":&lt;address&gt;   "firstname": &lt;firstname&gt;,   "lastname": &lt;lastname&gt;,   "hourly_rate":&lt;rate&gt;,   "years_xp":&lt;years&gt;,   "bio":&lt;bio&gt;, } </pre>                                                                                                                                                  |
| <b>Response Structure</b>  | <pre> {   "success": boolean,   "data": {     user {       "email": &lt;email&gt;,       "firstname": &lt;firstname&gt;,       "lastname": &lt;lastname&gt;,       "hourly_rate": &lt;rate&gt;,       "years_xp": &lt;years&gt;,       "bio": &lt;bio&gt;       "role": "cook",     },     "accessToken": &lt;token&gt;,     "refreshToken": &lt;token&gt;,   },   "error": {     "code": string, </pre> |

|  |                                                                           |
|--|---------------------------------------------------------------------------|
|  | <pre>"message": string, "details": string, }, "timestamp": string }</pre> |
|--|---------------------------------------------------------------------------|

## User Login

|                           |                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Description</b>        | User Login                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>API URL</b>            | /auth/login                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>HTTP Method</b>        | POST                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Format</b>             | JSON for all requests/responses                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Authentication</b>     | None                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Request Payload</b>    | <pre>{   "email": &lt;email&gt;,   "password": &lt;password&gt;, }</pre>                                                                                                                                                                                                                                                                                                                        |
| <b>Response Structure</b> | <pre>{   "success": boolean,   "data": {     "user": {       "email": &lt;email&gt;,       "firstname": &lt;firstname&gt;,       "lastname": &lt;lastname&gt;,       "address": &lt;address&gt;,       "role": &lt;role&gt;,     },     "accessToken": &lt;token&gt;   },   "error": {     "code": string,     "message": string,     "details": object null   },   "timestamp": string }</pre> |

## Core Feature Endpoints

### Cook Service Update

|                    |                                   |
|--------------------|-----------------------------------|
| <b>Description</b> | Update the cook portfolio details |
| <b>API URL</b>     | /services/{id}                    |
| <b>HTTP Method</b> | PUT                               |
| <b>Format</b>      | JSON for all requests/responses   |

|                           |                                                                                                                                                                                                                                        |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Authentication</b>     | Bearer token (Role: cook)                                                                                                                                                                                                              |
| <b>Request Payload</b>    | <pre>{   "bio": &lt;bio&gt;,   "years_xp": &lt;years&gt;,   "hourly_rate": &lt;rate&gt;, }</pre>                                                                                                                                       |
| <b>Response Structure</b> | <pre>{   "success": boolean,   "data": {     "cook_id": &lt;cook_id&gt;,     "updated_at": &lt;datetime&gt;   },   "error": {     "code": string,     "message": string,     "details": object null   },   "timestamp": string }</pre> |

## Payment

|                           |                                                                                                                                                                                                                                         |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Description</b>        | Get the payment URL from the gateway                                                                                                                                                                                                    |
| <b>API URL</b>            | /payment/checkout                                                                                                                                                                                                                       |
| <b>HTTP Method</b>        | POST                                                                                                                                                                                                                                    |
| <b>Format</b>             | JSON for all requests/responses                                                                                                                                                                                                         |
| <b>Authentication</b>     | Bearer Token (Role: customer)                                                                                                                                                                                                           |
| <b>Request Payload</b>    | <pre>{   "total_amount": &lt;amount&gt;,   "customer_id": &lt;customer_id&gt;, }</pre>                                                                                                                                                  |
| <b>Response Structure</b> | <pre>{   "success": boolean,   "data": {     "checkout_url": &lt;url&gt;,     "reference": &lt;reference&gt;   },   "error": {     "code": string,     "message": string,     "details": object null   },   "timestamp": string }</pre> |

## Create a booking request

|                           |                                                                                                                                                                                                                                                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Description</b>        | Customer booking request creation with payment verification                                                                                                                                                                                                                                                                       |
| <b>API URL</b>            | /booking                                                                                                                                                                                                                                                                                                                          |
| <b>HTTP Method</b>        | POST                                                                                                                                                                                                                                                                                                                              |
| <b>Format</b>             | JSON for all requests/responses                                                                                                                                                                                                                                                                                                   |
| <b>Authentication</b>     | Bearer Token (Role: customer)                                                                                                                                                                                                                                                                                                     |
| <b>Request Payload</b>    | <pre>{   "cook_id": &lt;cook_id&gt;,   "service_id": &lt;service_id&gt;,   "customer_id": &lt;customer_id&gt;,   "quantity":&lt;quantity&gt;,   "total_amount": &lt;total_amount&gt;,   "schedule_date": &lt;date&gt;,   "schedule_time": &lt;time&gt;,   "status": "pending",   "reference": &lt;payment_gateway_id&gt;, }</pre> |
| <b>Response Structure</b> | <pre>{   "success": boolean,   "data": {     "booking_id":&lt;booking_id&gt;,     "payment_id": &lt;payment_id&gt;   },   "error": {     "code": string,     "message": string,     "details": object null   },   "timestamp": string }</pre>                                                                                     |

## Cook booking management

|                        |                                                          |
|------------------------|----------------------------------------------------------|
| <b>Description</b>     | Cook booking management and schedule conflict resolution |
| <b>API URL</b>         | /booking/{id}/status                                     |
| <b>HTTP Method</b>     | PUT                                                      |
| <b>Format</b>          | JSON for all requests/responses                          |
| <b>Authentication</b>  | Bearer Token (Role: cook)                                |
| <b>Request Payload</b> | <pre>{   "status": &lt;status&gt; }</pre>                |

|                           |                                                                                                                                                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           | }                                                                                                                                                                                                                                 |
| <b>Response Structure</b> | <pre>{   "success": boolean,   "data": {     "booking_id":&lt;booking_id&gt;,     "status": "accepted"   },   "error": {     "code": string,     "message": string,     "details": object null   },   "timestamp": string }</pre> |

## Discovery Endpoints

### Browse Cooks

|                           |                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Description</b>        | Retrieves a list of available cooks with optional filters                                                                                                                                                                                                                                                                                                                                       |
| <b>API URL</b>            | /cook/all                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>HTTP Method</b>        | GET                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Format</b>             | JSON for all requests/responses                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Authentication</b>     | Bearer Token (Role: customer)                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Request Payload</b>    | none                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Response Structure</b> | <pre>{   "success": boolean,   "data": [ {     "id": &lt;id&gt;,     "firstname": &lt;firstname&gt;,     "lastname": &lt;lastname&gt;,     "bio": &lt;bio&gt;,     "hourly_rate": &lt;rate&gt;,     "years_xp": &lt;years&gt;,     "averageRating": &lt;float&gt;,   } ],   "error": {     "code": string,     "message": string,     "details": object null   },   "timestamp": string }</pre> |



## View the cook menu

|                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Description</b>        | Fetches the specific menu or services offered by a selected cook.                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>API URL</b>            | /cooks/{id}/services                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>HTTP Method</b>        | GET                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Format</b>             | JSON for all requests/responses                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Authentication</b>     | Bearer Token (Any role)                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Request Payload</b>    | none                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Response Structure</b> | <pre>{   "success": boolean,   "data": [ {     "id": &lt;id&gt;,     "cook_id":&lt;cook_id&gt;,     "title": &lt;title&gt;,     "description": &lt;description&gt;,     "ingredientList: &lt;ingredientList&gt;     "ingredientsCost": &lt;cost&gt;,     "est_prep_time":&lt;prep_time&gt;,     "image":&lt;image_url&gt;,     "Serving_size": &lt;serving_size&gt;   } ],   "error": {     "code": string,     "message": string,     "details": object null   },   "timestamp": string }</pre> |

## 5.3 Error Handling

### HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

## Error Code Examples

|           |                                                                                                                                                                                                                                                                                      |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Example 1 | <pre>json {   "success": false,   "data": null,   "error": {     "code": "AUTH-001",     "message": "Invalid credentials",     "details": "Email or password is incorrect"   },   "timestamp": "2024-01-28T10:30:00Z" }</pre>                                                        |
| Example 2 | <pre>{   "success": false,   "data": null,   "error": {     "code": "VALID-001",     "message": "Validation failed",     "details": {       "email": "Email is required",       "password": "Must be at least 8 characters"     }   },   "timestamp": "2024-01-28T10:30:00Z" }</pre> |
| Example 3 | <pre>{   "success": false,   "data": null,   "error": {     "code": "BUS-001",     "message": "Schedule Conflict",     "details": "Cook is already booked in that schedule."   },   "timestamp": "2024-01-28T10:30:00Z" }</pre>                                                      |
| Example 4 | <pre>{   "success": false,</pre>                                                                                                                                                                                                                                                     |

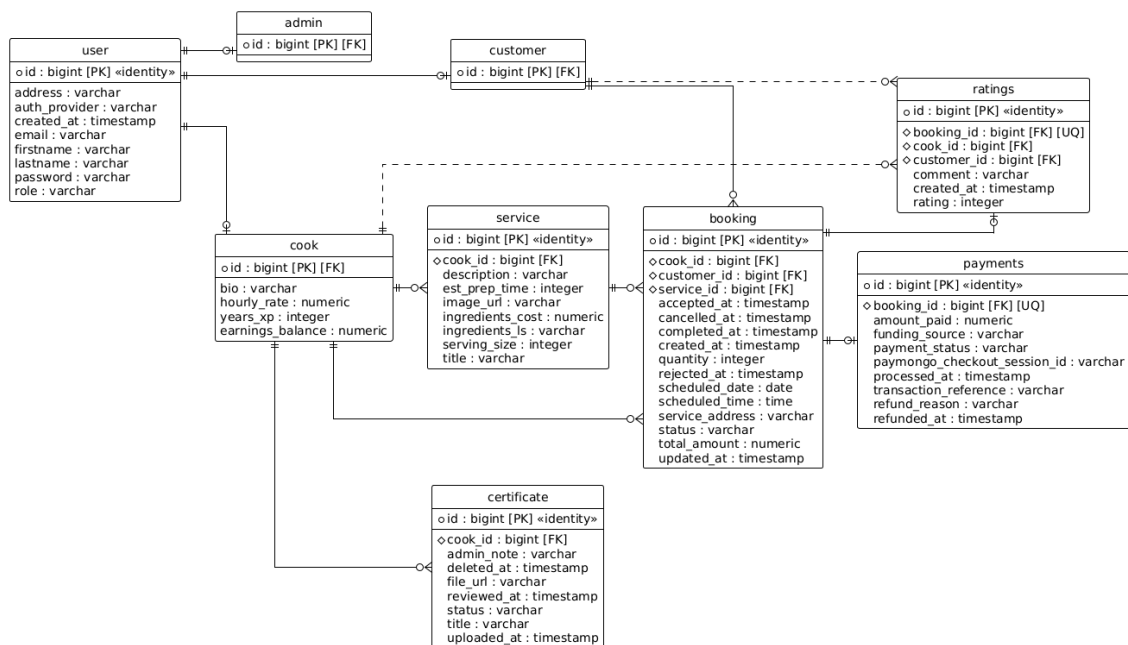
|  |                                                                                                                                                                                      |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre> "data": null, "error": {   "code": "PAY-001",   "message": "Payment Failed",   "details": "Payment Reference invalid or unpaid." }, "timestamp": "2024-01-28T10:30:00Z" </pre> |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUS-001: Schedule Conflict
- PAY-001: Payment Failed
- SYSTEM-001: Internal server error

## 6.0 DATABASE DESIGN

### 6.1 Entity Relationship Diagram



### Detailed Relationships:

- **One-to-One:** User ↔ Cook (A cook is a user)
- **One-to-One:** User ↔ Customer (A customer is a user)
- **One-to-Many:** Cook → Service (A cook can offer multiple menu services)
- **One-to-Many:** Cook → Certificate (A cook can have multiple certificates)
- **One-to-Many:** Customer → Booking (A customer can have multiple bookings over time)
- **One-to-Many:** Cook → Booking (A cook can be assigned to multiple bookings)
- **One-to-Many:** Service → Booking (A specific service can be booked multiple times)
- **One-to-One:** Booking → Rating (A booking can only be rated one time)
- **One-to-One:** Booking → Payment (A booking can only have one payment)

### Key Tables:

1. **users** - User accounts, authentication, and base class for customer/ cook
2. **cooks** - Extended profile for cooks
3. **customers** - Extended profile for customers
4. **Admin** - Extended profile for admin
5. **services** - Specific offerings/ services of cooks (e.g., "Filipino breakfast")
6. **certificate** - Cook's personal certificates
7. **booking** - Booking transactions
8. **ratings** - Rating of meal and service quality
9. **payments** - Records of payments for bookings

### Table Structure Summary:

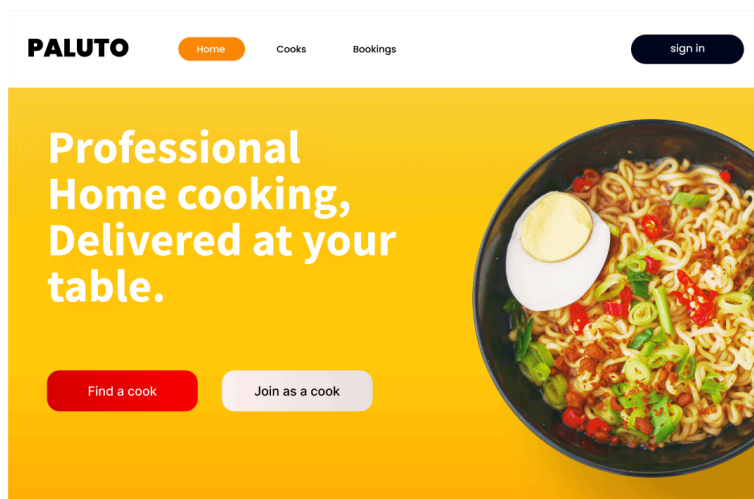
- **users:** id, email, firstname, lastname, address, password, role(customer/cook), auth\_provider
- **cooks:** user\_id, bio, hourly\_rate, years\_exp
- **customers:** user\_id,
- **admin:** admin\_id
- **services:** id, cook\_id, title, description, ingredients\_ls, est\_prep\_time, ingredients\_cost, image
- **certificates:** id, cook\_id, cert\_name, is\_verified, image

- **bookings:** id, customer\_id, cook\_id, service\_id, quantity, total\_amount, status, scheduled\_date, scheduled\_time, created\_at, updated\_at
- **ratings:** id, booking\_id, rating, comment
- **payments** - id, bookind\_id, reference, method, amount, status, paid\_at

## 7.0 UI/UX DESIGN

### 7.1 Web Application Wireframes

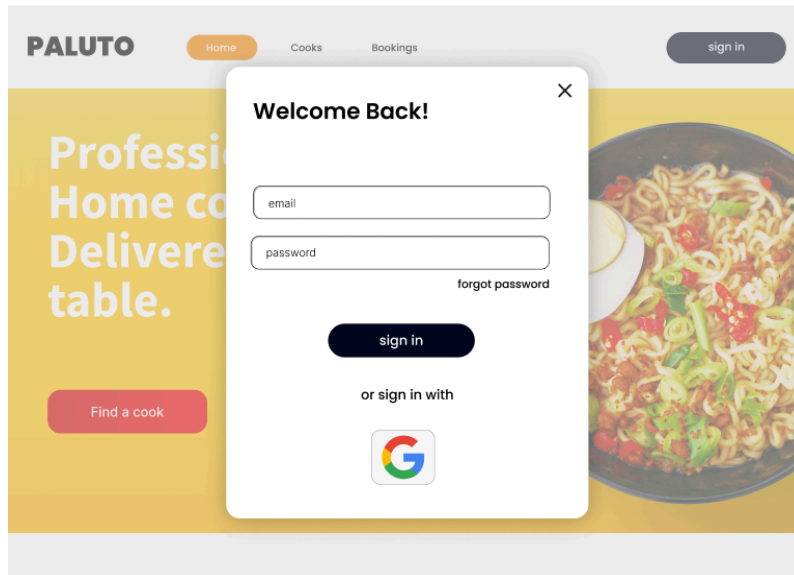
#### Landing page



Header: [Logo] [Nav bar] ["sign in" button]

Main: Tagline, ["Find a cook" button] ["join as a cook" button]

#### Login Modal



Login form, Google button, Forgot password, Close button

**Cooks page** (List of cooks)

**filter**Years experience Hourly rate 

Apply filter

**Cooks**

JD

John Doe

5 yrs

Bio: A home cook based on cebu city.

4.5 ★

Php 450

view menu

JD

John Doe

5 yrs

Bio: A home cook based on cebu city.

4.5 ★

Php 450

view menu

JD

John Doe

5 yrs

Bio: A home cook based on cebu city.

4.5 ★

Php 450

view menu

JD

John Doe

5 yrs

Bio: A home cook based on cebu city.

4.5 ★

Php 450

view menu

JD

John Doe

5 yrs

Bio: A home cook based on cebu city.

4.5 ★

Php 450

view menu

JD

John Doe

5 yrs

Bio: A home cook based on cebu city.

4.5 ★

Php 450

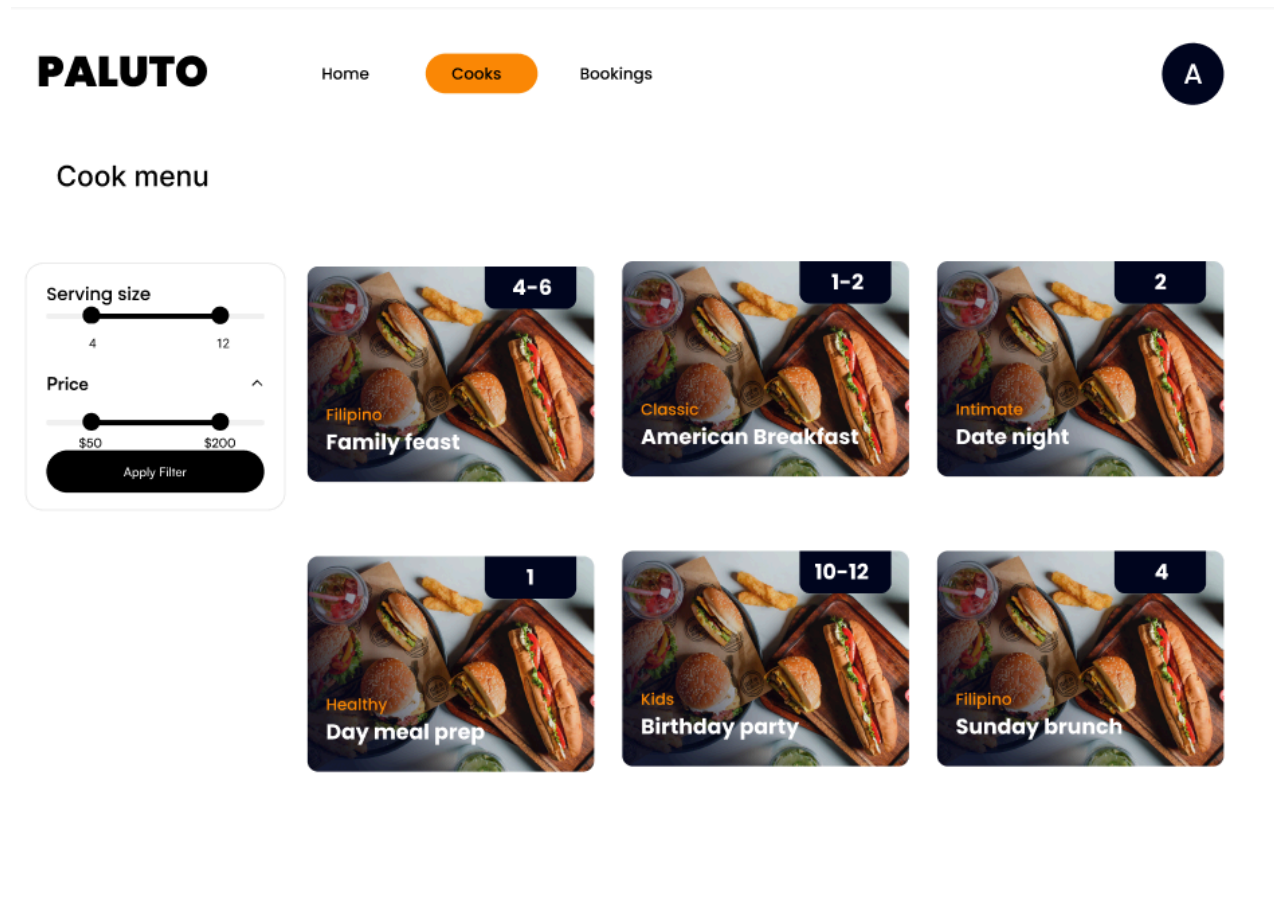
view menu

Header: [Logo] [Nav bar] [Profile]

Filter: Years of experience, hourly rate

Each cook card: Avatar, Full name, years of experience, Bio, rating, Hourly rate, “view menu” button

## Cook's menu



Header: [Logo] [Nav bar] [Profile]

Filter: Serving size, price

Each offering card: title, Serving size, offering image, title



## Meal details

**PALUTO**

Home

Cooks

Bookings

A

### Meal details



#### Ingredients:

Protein: 1kg Pork Belly, 500g Chicken Thighs.  
Vegetables: 1/4kg Squash, 100g Bitter Melon (Ampakaya), String Beans, Eggplant.  
Pantry Staples: Soy Sauce, Cane Vinegar, Peppercorns, Bay Leaves, Garlic, Shrimp Paste (Bagoong)

## Family Feast

A combination of common dishes served in most feast events in the Philippines.

Est. ingredient cost

Php 1, 000

Prep time

1 hour

Quantity

- 1 +

Book

Header: [Logo] [Nav bar] [Profile]

Meal image(Large), title, description, ingredients, ingredient cost, prep time, quantity stepper (default to 1), "Book" button

## Payment page

**PALUTO**

HomeCooksBookings

A

### Payment



#### Family Feast

DateTime

AM

Address

Payment Method

Php 1,450Pay now

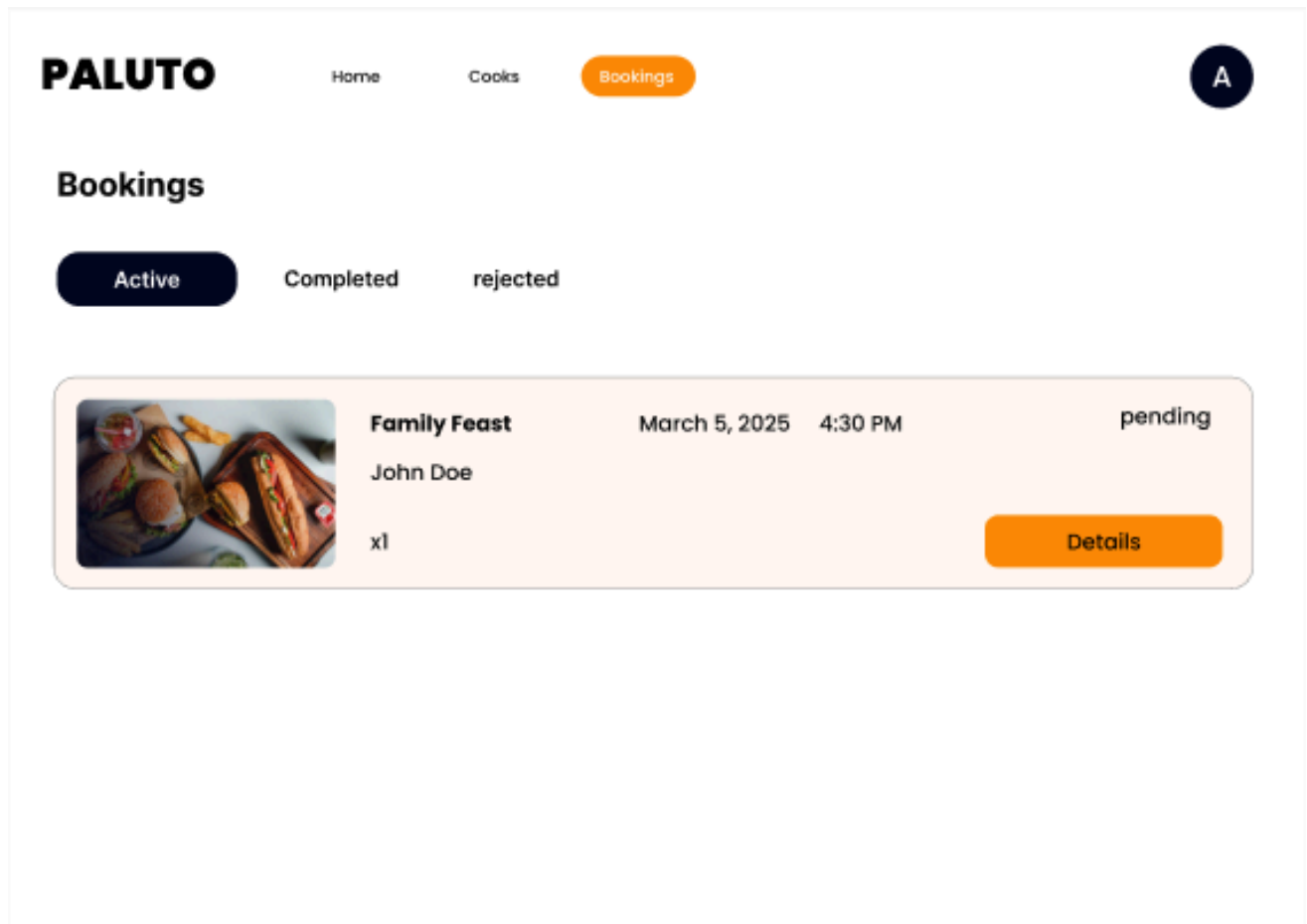
|                      |           |
|----------------------|-----------|
| Quantity             | x1        |
| Est. ingredient cost | Php 1,000 |
| Prep time            | 1 hour    |
| Cook rate/hr         | Php 450   |

Header: [Logo] [Nav bar] [Profile]

Summary: Quantity, ingredient cost, prep time, cook's rate per hour

Title, Date picker, time picker, address field, payment method field, total cost, "Pay now" button

## Bookings page (pending)

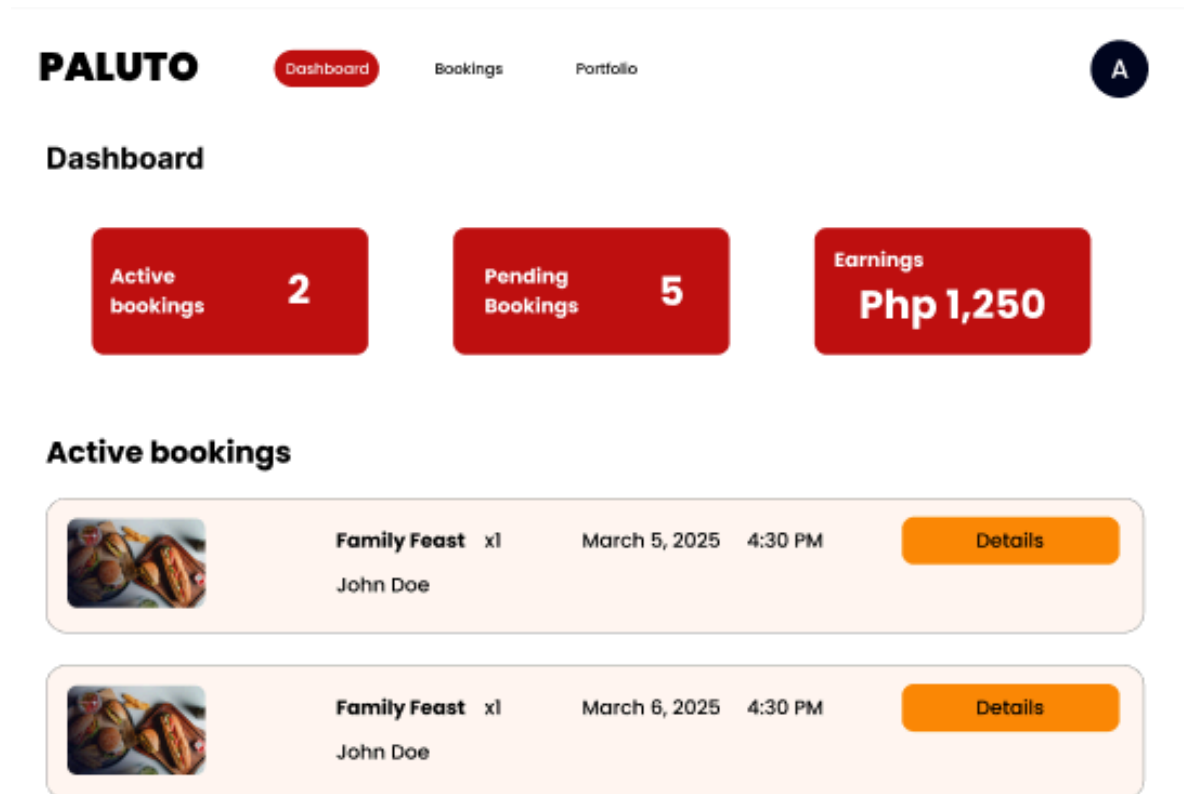


Header: [Logo] [Nav bar] [Profile]

Sub nav bar: Active, Completed, rejected

Each booking card: Meal image(small), title, cook's name, quantity, date, time, status, "Details" button.

## Cook dashboard



Header: [Logo] [Nav bar] [Profile]

KPI: Active bookings, Pending bookings, Earnings

Each active booking card: Meal image(small), title, quantity, customer name, date, time, "Details" button

## Cook bookings (pending)

**PALUTO**

Dashboard

Bookings

Portfolio

A

### Bookings

Pending

History



#### Family Feast

John Doe

x1

March 5, 2025 4:30 PM

Details

Reject

Accept



#### Family Feast

John Doe

x1

March 5, 2025 4:30 PM

Details

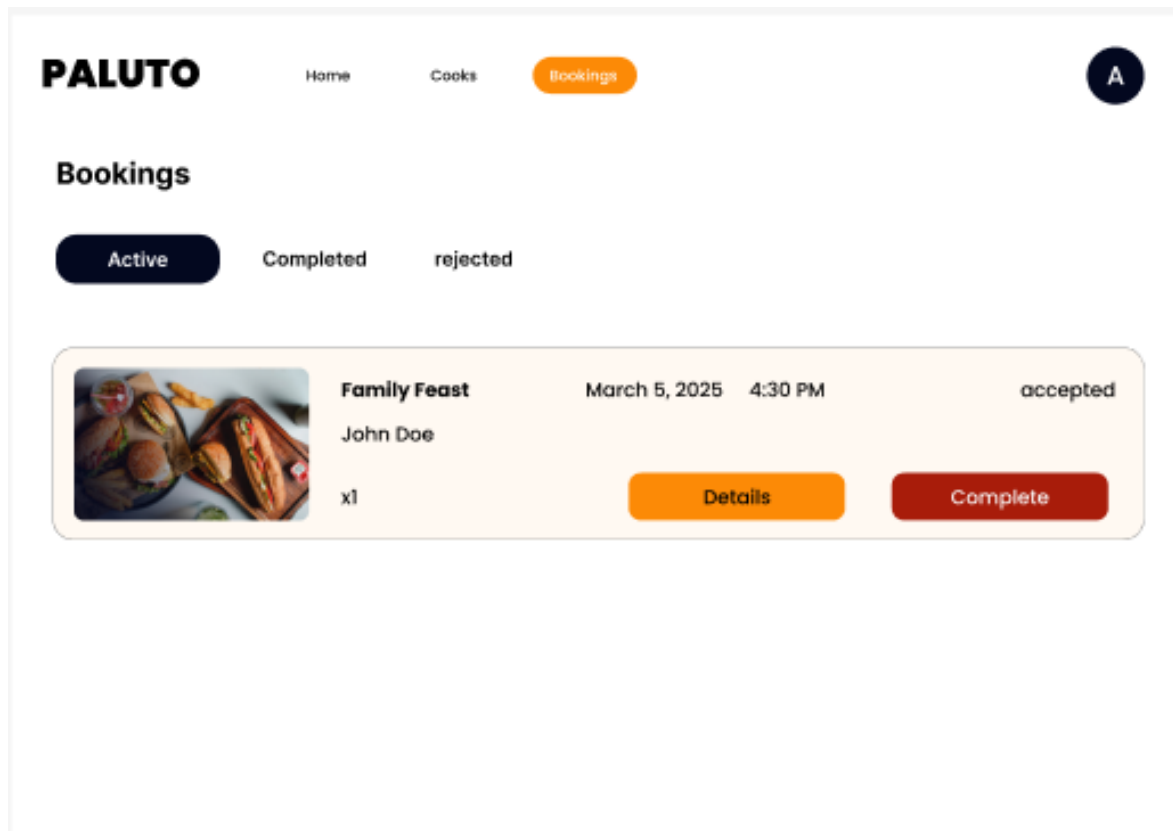
Reject

Accept

Header: [Logo] [Nav bar] [Profile]

Each booking card: meal image(small), title, customer name, quantity, date, time, "details" button, "Reject" button, "Accept" button.

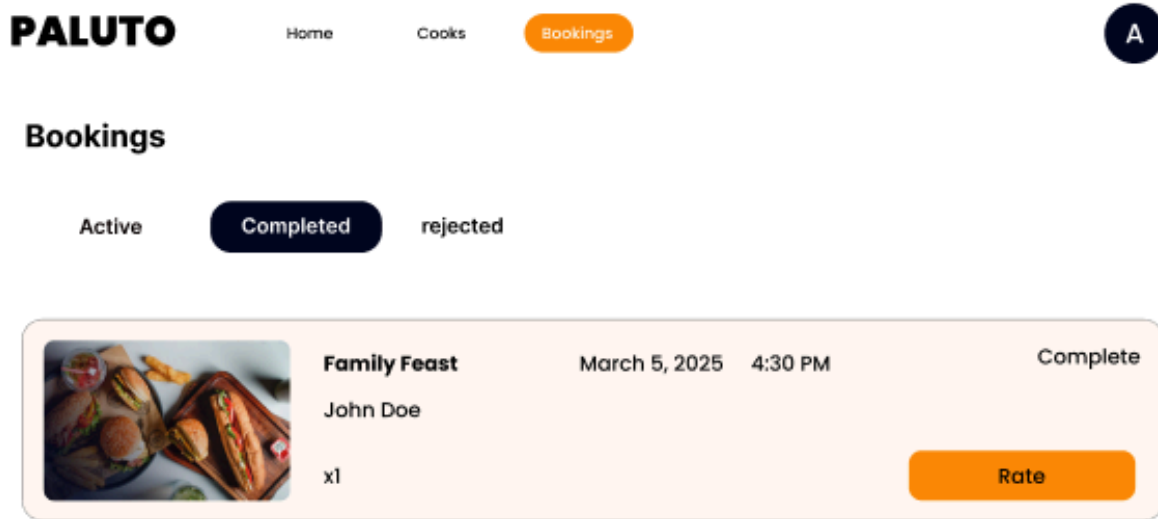
## Customer bookings (accepted bookings)



Header: [Logo] [Nav bar] [Profile]

Each booking card: meal image(small), title, customer name, quantity, date, time, status, "details" button, "Complete" button

## Customer bookings (completed bookings)



Header: [Logo] [Nav bar] [Profile]

Each booking card: meal image(small), title, customer name, quantity, date, time, status, "rate" button

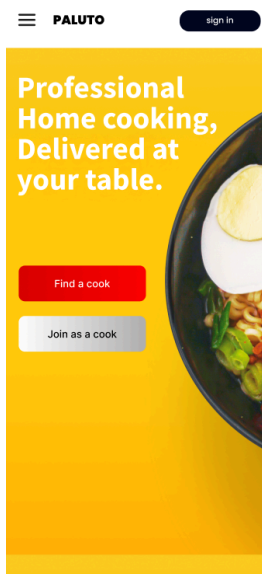
## 7.2 Mobile Application Wireframes

### Customer Bottom Navigation



Items: Home, Cooks, Bookings

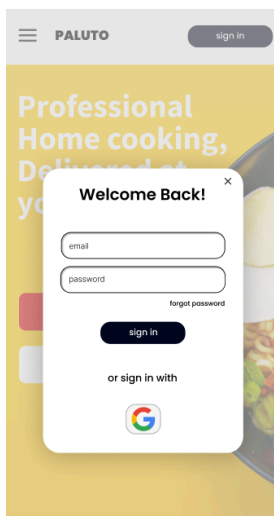
## Landing screen



Header: Hamburger menu, Logo, "sign in" button

Main: "Find a cook" button, "Join as a cook" button

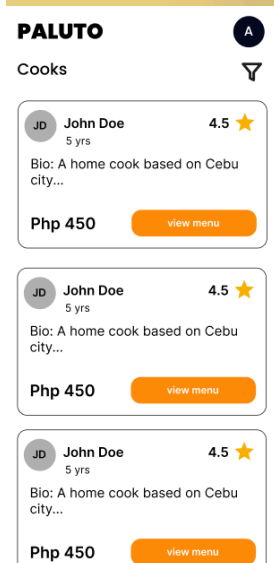
## Login Modal



Fields: Email, password

Forgot password, "sign in" button, sign in with "Google" button

## Cooks screen (List of cooks)



Header: Logo, Profile

Each cook card: Fullname, avatar, rating, bio, rate per hour, "view menu" button

filter icon, customer bottom navigation



## Cook's menu



Header: Logo, Profile

Each menu card: title, serving size, offering image (background)  
filter icon, bottom navigation

## Meal details

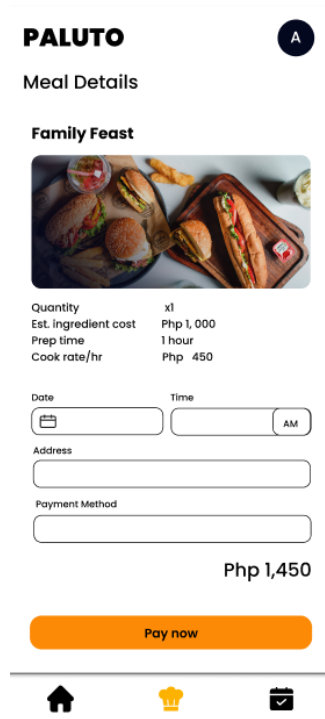


Header: Logo, Profile

title, prep time, meal image (medium), description, ingredients,  
ingredients cost, quantity stepper (Default to 1)

bottom navigation

## Payment screen



The Payment screen for the PALUTO app features a header with the logo and a profile icon. Below the header, the section is titled "Meal Details". A featured meal, "Family Feast", is displayed with an image. The meal details include: Quantity (x1), Estimated ingredient cost (Php 1,000), Prep time (1 hour), and Cook rate per hour (Php 450). Users can select a date and time (with AM/PM toggle) and enter an address. A payment method selection field is also present. The total cost is shown as "Php 1,450". A prominent orange "Pay now" button is at the bottom of the form. The bottom navigation bar contains icons for Home, Cart, and Profile.

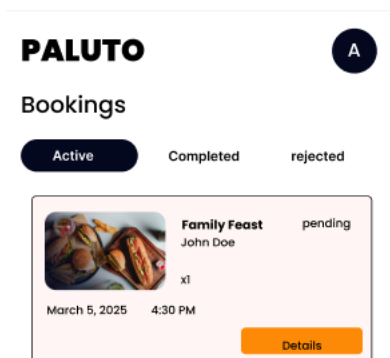
Header: Logo, Profile

Summary: Quantity, ingredient cost, prep time, cook's rate per hour

Title, Date picker, time picker, address field, payment method field, total cost, "Pay now" button

bottom navigation

## Bookings screen (pending)



The Bookings screen for the PALUTO app features a header with the logo and a profile icon. Below the header, the section is titled "Bookings". There are three tabs: "Active" (selected), "Completed", and "rejected". A booking card for "Family Feast" by "John Doe" is shown with a status of "pending". The card includes a meal image, quantity (x1), date and time (March 5, 2025, 4:30 PM), and a "Details" button. The bottom navigation bar contains icons for Home, Cart, and Profile.

Header: Logo, Profile

Screen nav bar: Active, Completed, rejected

Each booking card: Meal image(small), title, cook's name, quantity, date, time, status, "Details" button.

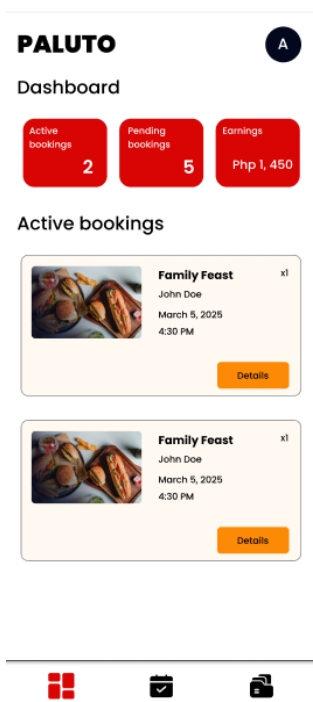
Bottom navigation

## Cook's bottom navigation



Items: Dashboard icon, Bookings icon, Portfolio icon

## Cooks dashboard



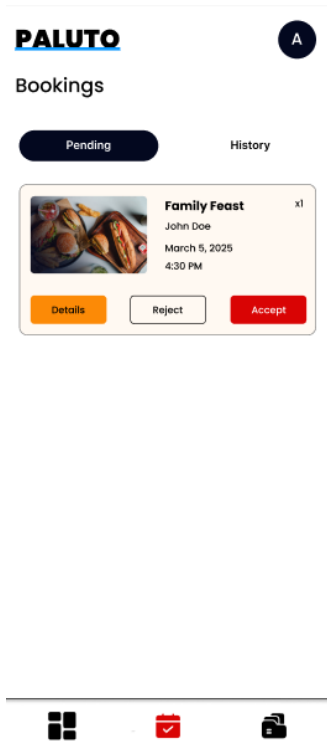
Header: Logo, Profile

KPI: Active bookings, Pending bookings, Earnings

Each active booking card: Meal image(small), title, quantity, customer name, date, time, "Details" button

Cook's bottom navigation

## Cooks bookings (pending)

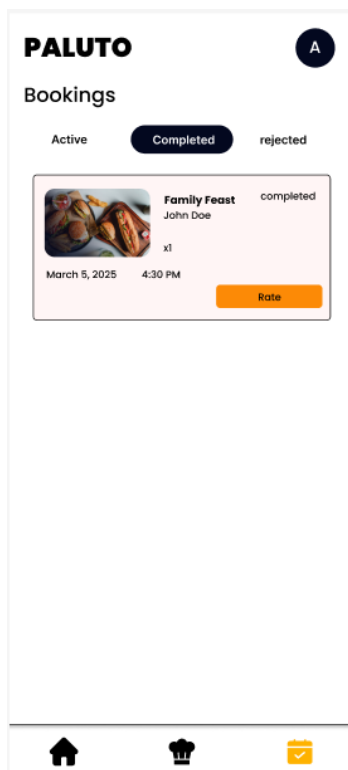


Header: Logo, Profile

Each booking card: meal image(small), title, customer name, quantity, date, time, “details” button, “Reject” button, “Accept” button.

Cook’s bottom navigation

## Customer bookings (accepted)

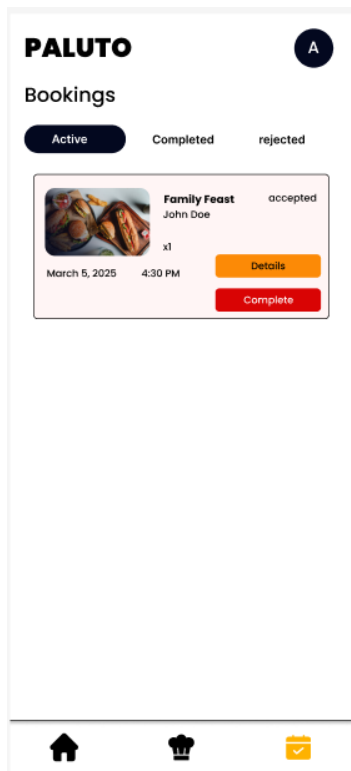


Header: Logo, Profile

Each booking card: meal image(small), title, customer name, quantity, date, time, status, “details” button, “Complete” button

Cook’s bottom navigation

## Customer bookings (completed)



Header: Logo, Profile

Each booking card: meal image(small), title, customer name, quantity, date, time, status, "rate" button

Cook's bottom navigation

## 8.0 PLAN

### 8.1 Project Timeline

#### Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

## **Phase 2: Backend Development (Week 3-4)**

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication & BCrypt password hashing implementation

Day 4: User management and /me endpoints

Day 5: Cook Portfolio CRUD operations

Week 4: Core Features

Day 1: Appointment/Booking transaction logic

Day 2: Service rating and file storage implementation

Day 3: Cooks and Menu filtering, Admin controls

Day 4: Error handling and validation

Day 5: API documentation and testing

## **Phase 3: Web Application (Week 5-6)**

Week 5: Frontend Foundation

Day 1: React setup

Day 2: Authentication pages (login, register)

Day 3: Cook portfolio (menu, certs) and Cook listing page (Public view)

Day 4: Menu listing, menu details page

Day 5: Appointment booking form implementation

Week 6: Complete Web Features

Day 1: Appointment management (Accept/Decline flow for Cooks)

Day 2: Booking history, Status updates, and Rating system

Day 3: Admin dashboard for Cook certification moderation

Day 4: Responsive design polish

Day 5: API integration and End-to-End testing

#### **Phase 4: Mobile Application (Week 7-8)**

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens (JWT Login/Register)

Day 3: Cook browsing and Portfolio view (XML Layouts)

Day 4: Appointment request and Calendar interaction

Day 5: Retrofit API service layer and JWT secure storage

Week 8: Complete Mobile App

Day 1: Role-aware UI

Day 2: Appointment and status management

Day 3: UI polish and role-based menu filtering

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

#### **Phase 5: Integration & Deployment (Week 9-10)**

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

**Milestones:**

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API (Auth & Booking) fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

**Critical Path:**

1. Authentication system, Google OAuth login, and Role-based access control (Week 3)
2. Cook portfolio API (Week 3-4)
3. Booking Transaction Engine - Handling payments and status updates (Week 4)
4. Conflict Resolution Logic - Ensuring the cook can't accept a booking with a similar timeline(Week 4)
5. Role-Aware Mobile Navigation



## 6. Cross-platform testing (Week 9)

### **Risk Mitigation:**

- Start with the simplest working version of each feature
- Test integration points early and often
- Keep a backup of the working versions
- Focus on core functionality before enhancements
- Database Safety First
- Limit External API Scope